

Q1. Write a Lex program to count identifiers in a C program.

Answer

```
%{
#include<stdio.h>
int count=0;
}%

%%
[a-zA-Z_][a-zA-Z0-9_]* {
    count++;
    printf("%s\n",yytext);
}

[ \t\n]+ ;
. ;
%%

int main()
{
    yylex();
    printf("\nTotal Identifiers=%d",count);
    return 0;
}
```

Q2. Write a Lex program to identify Integer, Floating Point and Exponential numbers.

Answer

```
%{
#include<stdio.h>
}%
```

```

%%
[0-9]+ {
    printf("%s : Integer\n",yytext);
}

[0-9]+\.[0-9]+ {
    printf("%s : Float\n",yytext);
}

[0-9]+[eE][+-]?[0-9]+ {
    printf("%s : Exponential\n",yytext);
}

[ \t\n]+ ;
. ;
%%

int main()
{
    yylex();
}

```

Q3. Write a Lex program to count Characters, Words and Lines.

Answer

```

%{
#include<stdio.h>

int chars=0;
int words=0;
int lines=0;
%}

%%

\n { lines++; chars++; }

[a-zA-Z0-9_]+ {
    words++;
    chars+=yyleng;
}

```

```

}

. {
    chars++;
}
%%

int main()
{
    yylex();

    printf("Characters=%d\n",chars);
    printf("Words=%d\n",words);
    printf("Lines=%d\n",lines);

    return 0;
}

```

Q4. Write a Lex program to count comments in C.

Answer

```

%{
#include<stdio.h>
int comments=0;
%}

%%

"//".* {
    comments++;
}

"/*"([^\*]|\*+[^*/])*\*+/" {
    comments++;
}

.\n ;
%%

int main()

```

```
{
    yylex();
    printf("Comments=%d",comments);
}
```

Q5. Write a Lex program to recognize Bangladesh Mobile Operators.

Answer

```
%{
#include<stdio.h>
%}

%%

017[0-9]{8} { printf("Grameenphone\n"); }

018[0-9]{8} { printf("Robi\n"); }

019[0-9]{8} { printf("Banglalink\n"); }

016[0-9]{8} { printf("Airtel\n"); }

015[0-9]{8} { printf("Teletalk\n"); }

.|\\n ;
%%

int main()
{
    yylex();
}
```

Q6. Write a Lex program to recognize "To Be" verbs.

Answer

```
%{
#include<stdio.h>
%}

%%
am|is|are|was|were|be|been|being
{
printf("%s is To-Be Verb\n",yytext);
}

.\n ;
%%

int main()
{
yylex();
}
```

Q7. Write a Lex program to classify Simple, Compound and Complex Sentences.

Answer

```
%{
#include<stdio.h>

int compound=0;
int complex=0;
%}

%%
and|but|or {
compound=1;
}

because|although|since|if|when {
complex=1;
}
```

```
.\n ;  
%%
```

```
int main()  
{  
yylex();
```

```
if(complex)  
printf("Complex Sentence");
```

```
else if(compound)  
printf("Compound Sentence");
```

```
else  
printf("Simple Sentence");  
}
```

Q8. Construct a Complete Lexer for C Programs.

Answer

```
%{  
#include<stdio.h>  
%}
```

```
KEYWORD int|float|char|if|else|for|while|return|void  
ID [a-zA-Z_][a-zA-Z0-9_]*  
NUM [0-9]+
```

```
%%
```

```
{KEYWORD}  
{  
printf("%s Keyword\n",yytext);  
}
```

```
{ID}  
{  
printf("%s Identifier\n",yytext);  
}
```

```

{NUM}
{
printf("%s Number\n",yytext);
}

"+|-|*|/"
{
printf("%s Operator\n",yytext);
}

";|'|"(')|"{'|"}"
{
printf("%s Delimiter\n",yytext);
}

[ \t\n]+ ;

. ;
%%

int main()
{
yylex();
}

```

SECTION B: FIRST and FOLLOW

Q9. Find FIRST and FOLLOW

Grammar:

```

E → TR
R → +TR | ε
T → FY
Y → *FY | ε
F → (E) | id

```

FIRST

$\text{FIRST}(E) = \{ (, \text{id} \}$
 $\text{FIRST}(R) = \{ +, \epsilon \}$
 $\text{FIRST}(T) = \{ (, \text{id} \}$
 $\text{FIRST}(Y) = \{ *, \epsilon \}$
 $\text{FIRST}(F) = \{ (, \text{id} \}$

FOLLOW

$\text{FOLLOW}(E) = \{ \$,) \}$
 $\text{FOLLOW}(R) = \{ \$,) \}$
 $\text{FOLLOW}(T) = \{ +, \$,) \}$
 $\text{FOLLOW}(Y) = \{ +, \$,) \}$
 $\text{FOLLOW}(F) = \{ *, +, \$,) \}$

Q10. Is this Grammar Ambiguous?

$E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow \text{id}$

Answer

YES

Because:

$\text{id} + \text{id} * \text{id}$

can produce more than one parse tree.

Therefore:

Grammar is Ambiguous.

Non-Ambiguous Grammar

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow \text{id}$



VIVA QUESTIONS & ANSWERS

Q1. What is a Compiler?

Answer

A compiler is a program that translates source code into machine code.

Q2. What is Lexical Analysis?

Answer

The first phase of compiler that converts source code into tokens.

Q3. What is a Token?

Answer

A token is the smallest meaningful unit of a program.

Example:

```
int a=5;
```

Tokens:

```
int  
a  
=  
5  
;
```

Q4. What is Lex?

Answer

Lex is a tool used to generate lexical analyzers automatically.

Q5. What is Flex?

Answer

Flex is the modern version of Lex.

Q6. What is yytext?

Answer

yytext stores the matched lexeme.

Example:

input = hello

yytext = hello

Q7. What is yyleng?

Answer

Stores the length of matched string.

Example:

hello

yyleng=5

Q8. What is a Lexeme?

Answer

The actual sequence of characters matched by a token.

Example:

count

is a lexeme of token Identifier.

Q9. Difference Between Token and Lexeme?

Answer

Token	Lexeme
Category	Actual Value
Identifier	count
Number	100

Q10. What is Regular Expression?

Answer

A pattern used to describe strings.

Example:

[a-z] +

matches lowercase words.

Q11. What is CFG?

Answer

Context Free Grammar.

Used to define programming language syntax.

Q12. What are Components of CFG?

Answer

1. Non-terminal
 2. Terminal
 3. Production
 4. Start Symbol
-

Q13. What is FIRST Set?

Answer

FIRST(X) contains terminals that appear first in strings derived from X.

Q14. What is FOLLOW Set?

Answer

FOLLOW(X) contains terminals that can appear immediately after X.

Q15. What is Ambiguous Grammar?

Answer

A grammar that generates more than one parse tree for the same string.

Q16. What is Parse Tree?

Answer

A tree representation of derivation.

Q17. What is NFA?

Answer

Non-deterministic Finite Automata.

Can have multiple transitions for same input.

Q18. What is DFA?

Answer

Deterministic Finite Automata.

Only one transition per input symbol.

Q19. Difference Between DFA and NFA?

DFA	NFA
One transition	Multiple transitions
Faster	Easier to design

Q20. Why Convert NFA to DFA?

Answer

Because DFA is easier and faster for implementation in lexical analyzers.

Most Important Viva Questions (Teacher Favorites)

1. What is Compiler?
2. What is Token?
3. What is Lexeme?
4. What is Lex?
5. What is yytext?
6. What is yyleng?
7. Difference between Token and Lexeme?
8. What is CFG?
9. What is FIRST?
10. What is FOLLOW?
11. What is Ambiguous Grammar?
12. Difference between DFA and NFA?
13. What is Parse Tree?
14. What is Regular Expression?
15. Explain Lex Program Structure.

If you can confidently answer these 15 viva questions and write the 8 Lex programs above, you'll cover the majority of what typically appears in a Compiler Lab final.